Планирование: Multi-Level Feedback Queue

В этой лекции мы обсудим о проблемах разработки одного из самых известных подходов к планированию, который называется Multi-Level Feedback Queue (MLFQ). Впервые MLFQ-планировщик был описан в 1962 году Fernando J. Corbató в системе, называемой Compatible Time-Sharing System (CTSS). Эта работа (в том числе и поздние работы над Multics) впоследствии были представлены к награде Turing Award. Планировщик был впоследствии усовершенствован и приобрел облик, который можно встретить уже в некоторых современных системах.

Алгоритм MLFQ пытается решить 2 фундаментальные пересекающиеся проблемы. **Во-первых** он пытается оптимизировать время оборота, которое как мы рассматривали в предыдущей лекции, оптимизируется методом запуска в начале очереди наиболее коротких задач. Однако, ОС не знает как долго будет работать тот или иной процесс, а это необходимое знание для работы алгоритмов SJF, STCF. **Во-вторых** MLFQ пытается сделать систему отзывчивой для пользователей (например для таких, которые сидят и пялятся в экран в ожидании завершения задачи) и таким образом минимизировать время отклика. К сожалению, алгоритмы наподобие RR уменьшают время отклика, но крайне плохо влияют на метрику времени оборота. Отсюда наша проблема: Как проектировать планировщик, который будет отвечать нашим требованиям и при этом ничего не знать о характере процесса в общем? Как планировщик сможет изучать характеристики задач, которые он запускает и таким образом принимать лучше решения о планировании?

Суть проблемы: Как планировать постановку задач без идеального знания?
Как разрабатывать планировщик который одновременно минимизирует время отклика для интерактивных задач и при этом минимизировать время оборота без заведомого знания времени исполнения задачи?

Заметка: обучаемся по предыдущим событиям
Очередь MLFQ является прекрасным примером системы, которая обучается на прошедших событиях, чтобы предсказать будущее. Подобные подходы часто встречаются в ОС (И многих других отраслях в информатике, включая ветки предсказаний в оборудовании и алгоритмы кэширования). Подобные походы срабатывают когда у задач есть поведенческие фазы и таким образом они предсказуемы. Однако с такой техникой следует быть осторожным, потому что предсказания очень легко могут оказаться неправильными и привести систему к принятию худших решений, нежели были бы без знаний вообще.

MLFQ: Basic Rules

Рассмотрим базовые правила алгоритма MLFQ. И хотя реализаций этого алгоритма существует несколько, базовые подходы похожи.

В той реализации, которую мы будем рассматривать, в MLFQ будет несколько отдельных очередей, каждая из которых будет иметь разный приоритет. В любое время, задача, готовая к исполнению находится в одной очереди. MLFQ использует приоритеты, чтобы решить, какую задачу запускать на исполнение, т.е. задача с более высоким приоритетом (задача из очереди с высшим приоритетом) будет запущена в первую очередь.

Несомненно в конкретной очереди может находиться более одной задачи, таким образом у них будет одинаковый приоритет. В этом случае будет использоваться механизм RR для планирования запуска среди этих задач.

Таким образом мы приходим к двум базовым правилам для MLFQ:

- Rule1: Если приоритет(А) > Приоритет(Б), будет запущена задача А (Б не будет)
- Rule2: Если приоритет(А) = Приоритет(Б), АиБ запускаются с использованием RR

Исходя из вышеперечисленного, ключевым элементом к планированию MLFQ являются приоритеты. Вместо того чтобы задавать фиксированный приоритет каждому заданию, MLFQ изменяет его приоритет в зависимости от наблюдаемого поведения. Например, если задача постоянно бросает работу на CPU в ожидании ввода с клавиатуры, MLFQ будет поддерживать приоритет процесса на высоком уровне, потому что именно так должен работать интерактивный процесс. Если же напротив задача постоянно и интенсивно использует CPU в течение долгого периода, MLFQ будет понижать его приоритет. Таким образом MLFQ будет **изучать** поведение процессов в момент их работы и использовать поведенческую **историю** для предсказания **будущего** поведения.

Нарисуем пример того как могли бы выглядеть очереди в некоторый момент времени и тогда получится что-то вроде этого:

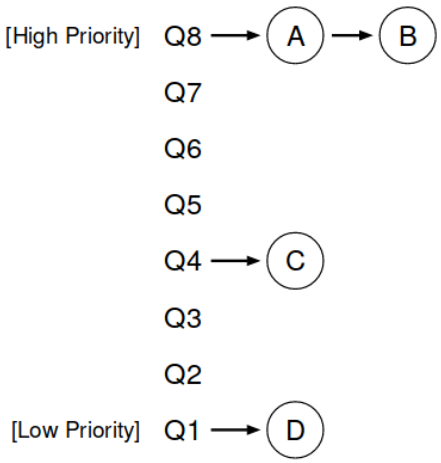


Figure 8.1: **MLFQ Example**

В данной схеме 2 процесса A и B находятся в очереди с наивысшим приоритетом. Процесс C где-то посередине, а процесс D в самом конце очереди. Согласно приведенным выше описаниям алгоритма MLFQ планировщик будет исполнять задачи только с наивысшим приоритетом согласно RR, а задачи C,D будут не у дел.

Естественно статический снапшот не даст полную картину того как работает MLFQ. Важно понимать, как именно меняется картина с течением времени.

Попытка 1: Как изменять приоритет

В этот момент необходимо решить как MLFQ будет менять уровень приоритета задачи (и таким образом положения задачи в очереди) по ходу ее жизненного цикла. Для этого необходимо держать в голове рабочий процесс: некоторое количество интерактивных задач с коротким временем работы (и таким образом частое освобождение CPU) и несколько долгих задач, которые используют CPU все свое рабочее время, при этом время отклика для таких задач не важно. И таким образом можно сделать первую попытку реализовать алгоритм MLFQ со следующими правилами:

- Rule3: Когда задача поступает в систему, она помещается в очередь с наивысшим приоритетом.
- Rule4a: Если задача использует целиком отведенное ей временное окно, то ее приоритет понижается.
- Rule4b: Если Задача освобождает CPU до истечения своего временного окна, то она остается с прежним приоритетом.

Пример 1: Одиночная долгоработающая задача

Как видно в этом примере, задача при поступлении ставится с наивысшим приоритетом. После временного окна в 10мс процесс понижается в приоритете планировщиком. После следующего временного окна задача наконец понижается до низшего приоритета в системе, где и остается.

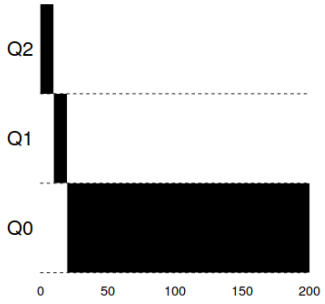


Figure 8.2: **Long-running Job Over Time**

Пример 2: Подвезли короткую задачу

Теперь посмотрим пример того, как MLFQ попытается приблизиться к SJF. В этом примере две задачи: A, которая является долгоиграющей задачей постоянно занимающей

CPU и B, которая является короткой интерактивной задачей. Предположим, что A уже работала некоторое время к моменту когда поступила задача B.

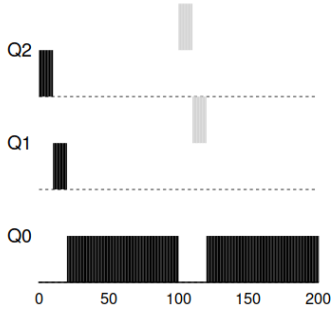


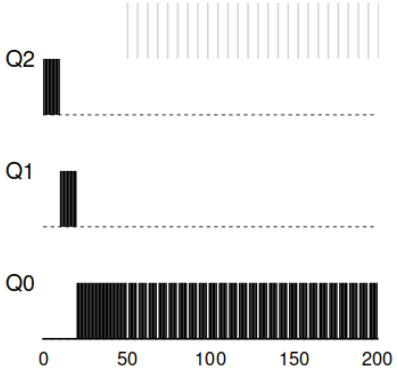
Figure 8.3: **Along Came An Interactive Job**

На данном графике видны результаты сценария. Задача A, как и любая задача, использующая CPU оказалась в самом низу. Задача B прибудет во время T=100 и будет помещена в очередь с наивысшим приоритетом. Поскольку время ее работы невелико, то она завершится до того как достигнет последней очереди.

Из этого примера следует понять главную цель алгоритма: поскольку алгоритм не знает длинная задача или короткая, то в первую очередь он предполагает, что задача короткая и выдает ей наивысший приоритет. Если это действительно короткая задача, то она выполнится быстро, иначе если это длинная задача, то она будет медленно двигаться в приоритете вниз и вскоре докажет, что она действительно долгая задача, которая не требует отклика.

Пример 2: Что же о вводе-выводе?

Теперь взглянем на пример с вводом-выводом. Как утверждалось в правиле 4b, если процесс освобождает процессор не использовав полностью его процессорное время, то он остается на прежнем уровне приоритета. Намерения этого правила довольно просты - если интерактивное задание выполняет много операций ввода-вывода, например ожидая от пользователя нажатий клавиши или мыши, такое задание будет освобождать процессор раньше отведенного окна. Мы не хотели бы хотели опускать такое задание по приоритету и таким образом оно останется на прежнем уровне.



Этот пример показывает как будет работать алгоритм с такими процессами - интерактивное задание B, которое нуждается в CPU только на 1мс перед выполнением процесса ввода-вывода и долгое задание A, которое все свое время использует CPU. MLFQ держит процесс B с наивысшим приоритетом, поскольку он все время продолжает освобождать CPU. Если B - интерактивное задание, то алгоритм в таком случае достиг своей цели запускать интерактивные задачи быстро.

Проблемы с текущим алгоритмом MLFQ

В предыдущих примерах мы построили базовый вариант MLFQ. И кажется что он выполняет свою работу хорошо и честно, распределяя процессорное время честно между долгими задачами и позволяя коротким задачам или задачам интенсивно обращающимся к вводу-выводу отрабатывать быстро. К сожалению такой подход содержит несколько серьезных проблем.

Во-первых, проблема голода: если в системе будет множество интерактивных задач, то они будут потреблять все процессорное время и таким образом ни одно долгое задание не получит возможности исполняться (они голодают).

Во-вторых умные пользователи могли бы писать свои программы так, чтобы обмануть планировщик. Обман кроется в том чтобы сделать что-то такое, чтобы заставить планировщик выдавать процессу больше процессорного времени. Алгоритм, который описан выше вполне уязвим к подобным атакам: перед тем как окно времени практически кончилось нужно выполнить операцию ввода-вывода (к какому-то не важно какому файлу) и таким образом освободить CPU. Подобное поведение позволит оставаться в той же самой очереди и снова получить больший процент процессорного времени. Если сделать это правильно (например исполняться 99% времени окна перед освобождением CPU), такая задача попросту может монополизировать процессор.

Наконец, программа может менять свое поведение со временем. Те задачи, которые использовали CPU могут стать интерактивными. В нашем примере подобные задачи не получат должного обращения от планировщика, так как получили бы другие (изначальные) интерактивные задачи.

Вопрос к залу: какие атаки на планировщик можно было совершать в современном мире?

Попытка 2: Повышение приоритета

Попробуем поменять правила и посмотрим получится ли избежать проблем с голоданием. Что бы мы могли сделать для того чтобы гарантировать чтобы связанные с CPU задачи получали свое время (даже если и не долгое).

В качестве простого решения проблемы можно предложить периодически повышать приоритет всех таких задач в системе. Существует множество способов достижения этого, попробуем воплотить в качестве примера что-то простое: переводить сразу все задачи в наивысший приоритет, отсюда новое правило:

- Rule5: По прошествии некоторого периода S перевести все задачи в системе в наивысшую очередь.

Наше новое правило решает две проблемы сразу. Во-первых процессы гарантированно не голодают: задачи, находящиеся в высшей очереди будут делить процессорное время согласно алгоритму RR и таким образом все процессы получат процессорное время. Во-вторых, если какой-то процесс, который ранее использовал только процессор, становится интерактивным, то он будет оставаться в очереди с высшим приоритетом после того как единожды получит повышение приоритета до наивысшего.

Рассмотрим пример. В этом сценарии рассмотрим один процесс, использующий CPU и два интерактивных, коротких процесса.

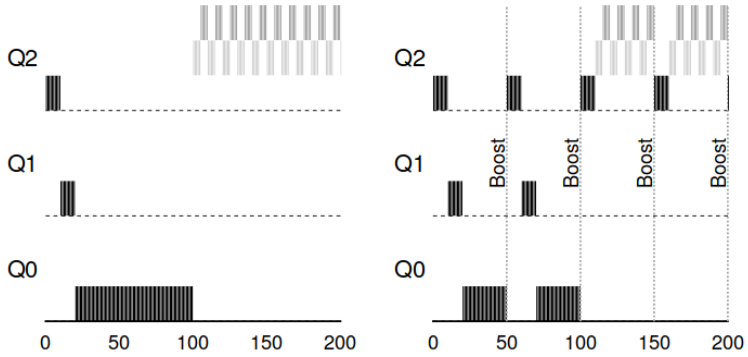


Figure 8.5: **Without (Left) and With (Right) Priority Boost**

Слева на рисунке рисунок демонстрирует поведение без повышения приоритета, и таким образом долгая задача начинает голодать после прибытия в систему двух интерактивных задач. На рисунке справа каждые 50мс выполняется повышение приоритета и таким образом все процессы гарантированно получают процессорное время и будут периодически запускаться. 50мс в данном случае взято для примера, реально это число несколько больше.

Очевидно, что добавление времени периодического повышения S приводит к закономерному вопросу: какое значение должно быть выставлено? Один из заслуженных системных инженеров John Ousterhout называл подобные величины в системах как voo-doo константа, поскольку они в некотором роде требовали черной магии для корректного выставления. И к сожалению S имеет такой аромат. Если выставить значение слишком большим - долгие задачи начнут голодать. А если выставить слишком низкое значение, интерактивные задачи не получат должного процессорного времени.

Попытка 3: Лучший учет

Теперь у нас есть еще одна проблема, которую необходимо решить: как не позволять обманывать наш планировщик? Виноватыми в этой возможности оказываются правила 4a, 4b, которые позволяют заданию сохранять приоритет освобождая процессор до истечения выделенного времени. Как же с этим справиться?

Решением в этом случае можно считать лучший учет времени CPU на каждом уровне MLFQ. Вместо того чтобы забывать время, которое программа использовала

процессор за отведенный промежуток, следует учитывать и сохранять его. После того как процесс израсходовал отведенное ему время следует понижать его до следующего уровня приоритета. Теперь не важно как процесс будет использовать свое время - как постоянно вычисляющий на процессоре или как множество вызовов. Таким образом следует переписать правило 4 к следующему виду:

- Rule4: После того как задача израсходовала выделенное ей время в текущей очереди (независимо от того сколько раз она освобождала CPU) приоритет такой задачи понижается (она двигается вниз очереди).

Посмотрим на пример:

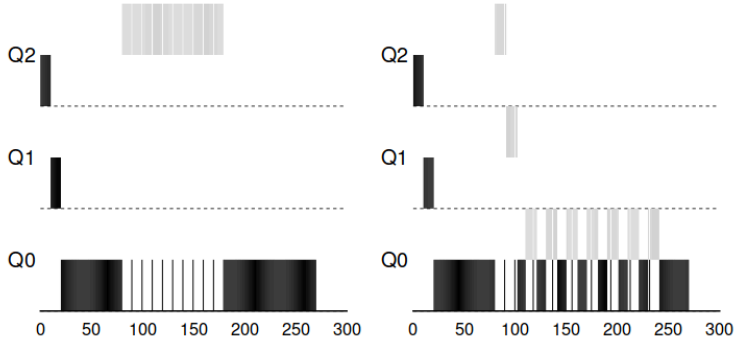


Figure 8.6: **Without (Left) and With (Right) Gaming Tolerance**

На рисунке показано что случается, если попробовать обмануть планировщик как если бы было с предыдущими правилами 4a, 4b получится результат слева. С новым правилом - результат справа. До защиты любой процесс мог вызвать I/O до завершения и таким образом доминировать на CPU, после включения защиты, независимо от поведения I/O, он будет все равно опускаться в очереди вниз и таким образом не сможет нечестно завладеть ресурсами CPU.

Улучшаем MLFQ и прочие проблемы

С приведенными выше улучшениями возникают новые проблемы: один из главных вопросов - как параметризировать подобный планировщик? Т.е. Сколько должно быть очередей? Каков должен быть размер окна работы программы в пределах очереди? Как часто следует повышать приоритет программы для того чтобы избежать голодания и принять во внимание изменение поведения программы? На эти вопросы нет простого ответа и только эксперименты с нагрузками и последующее конфигурирование планировщика может привести к некоему удовлетворительному балансу.

Например большинство реализаций MLFQ позволяют назначать различные временные интервалы различным очередям. Высокоприоритетным очередям обычно назначаются короткие интервалы. Эти очереди состоят из интерактивных задач, переключение между которыми довольно чувствительно и должно занимать 10 или меньше мс. В противовес низкоприоритетные очереди состоят из долгих задач, использующих CPU. И в этом случае длинные временные интервалы подходят очень хорошо (100мс).

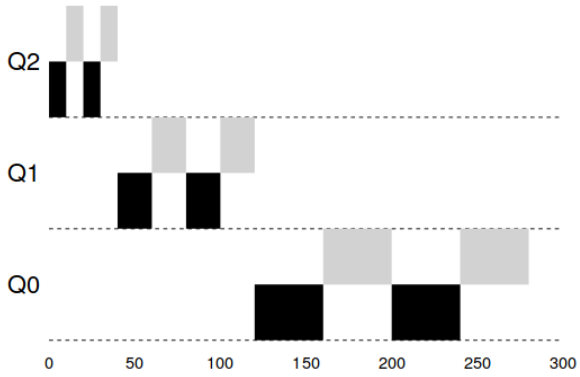


Figure 8.7: **Lower Priority, Longer Quanta**

В этом примере есть 2 задачи, которые проработали в высокоприоритетной очереди 20 мс, разбитые на 10мс окна. 40мс в средней очереди (окно в 20мс) и в низкоприоритетной очереди временное окно стало 40мс, где задачи завершили свою работу.

Реализация MLFQ в ОС Solaris - класс планировщиков, разделяющих по времени. Планировщик предоставлят набор таблиц, которые определяют в точности, как должен меняться приоритет процесса с течением его жизни, каким должен быть размер выделяемого окна и как часто нужно поднимать приоритеты задачи. Администратор системы может взаимодействовать с этой таблицей и заставлять планировщик вести себя по-другому. По-умолчанию в этой таблице 60 очередей с постепенным увеличением размера окна с 20мс (высокий приоритет) до нескольких сотен мс (низший приоритет), а также с бустом всех задач раз в секунду.

Другие MLFQ планировщики не используют таблицу или какие-то конкретные правила, которые описаны в этой лекции, напротив они вычисляют приоритеты используя математические формулы. Так например планировщик в FreeBSD использует формулу для вычисления текущего приоритета задачи, основываясь на том сколько процесс использовал CPU. В дополнение, использование CPU со временем загнивает, и таким образом повышение приоритета происходит несколько иначе чем описано выше. Это так называемые decay алгоритмы. С версии 7.1 в FreeBSD используется планировщик ULE.

Наконец, многие планировщики имеют другие особенности. Например некоторые планировщики резервируют высшие уровни для работы операционной системы и таким образом никакой пользовательский процесс не сможет получить наивысший приоритет в системе. Некоторые системы позволяют давать советы для того чтобы помочь планировщику корректно выставлять приоритеты. Так например с помощью команды nice можно увеличивать или уменьшать приоритет задачи и таким образом повышать или понижать шансы программы на процессорное время.

MLFQ: Итоги

Мы описали подход к планированию, который называется MLFQ. Его название заключено в принципе работы - он имеет несколько очередей и использует обратную связь для определения приоритета задачи.

Итоговый вид правил будет следующим:

- Rule1: Если приоритет(А) > Приоритет(Б), будет запущена задача А (Б не будет)
- Rule2: Если приоритет(А) = Приоритет(Б), АиБ запускаются с использованием RR
- Rule3: Когда задача поступает в систему, она помещается в очередь с наивысшим приоритетом.
- Rule4: После того как задача израсходовала выделенное ей время в текущей очереди (независимо от того сколько раз она освобождала CPU) приоритет такой задачи понижается (она двигается вниз очереди).
- Rule5: По прошествии некоторого периода S перевести все задачи в системе в наивысшую очередь.

MLFQ интересен по следующей причине - вместо того чтобы требовать знание о природе задачи заранее, алгоритм изучает прошлое поведение задачи и выставляет приоритеты соответствующе. Таким образом он старается усидеть сразу на двух стульях - достичь производительности для маленьких задач (SJF, STCF) и честно запускать долгие, загружающие CPU задания. Поэтому многие системы, включая BSD и их производные, Solaris, Windows, Mac используют в качестве планировщика некоторую форму алгоритма MLFQ в качестве базовой основы.

Дополнительные материалы:

https://manpages.debian.org/stretch/manpages/sched.7.en.html
https://en.wikipedia.org/wiki/Scheduling_(computing)
https://pages.lip6.fr/Julia.Lawall/atc18-bouron.pdf
https://www.usenix.org/legacy/event/bsdcon03/tech/full_papers/roberson/roberson.pdf
https://chebykin.org/freebsd-process-scheduling